



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester(2025-26)
Faculty Name	Ms. P. Swethanagasri
Student Name	SURABHI ANUSHKA
Roll Number	2520090168
Branch	CSIT
Course Outcome	1

Case Study Topic: MediFlow Hospital — appointment-driven patient indexing.

Work Done Summary:

A Binary Search Tree (BST) was created using the patient IDs 20, 30, 35, 40, 45, 50, 60, 65, 70, 75, 80, 85, and 90, which arrived in nearly sorted order, causing the tree to become right-skewed. Three deletion operations were then performed in sequence: removing patient IDs 30, 70, and 50, each of which had a single child and was replaced by that child. After the deletions, the BST contained 10 nodes and remained right-skewed with a height of 9 edges. Although the current lookup time remains within the required SLA, the structure demonstrates how a plain BST can become inefficient with sorted input, making a self-balancing tree such as an AVL Tree or Red-Black Tree a better choice for maintaining fast patient record searches.

Implementation Details :

- Implemented a Binary Search Tree (BST) to store patient IDs.
- Inserted 13 patient IDs in the given appointment order: 20, 30, 35, 40, 45, 50, 60, 65, 70, 75, 80, 85, and 90.
- Each new ID was compared with existing nodes and inserted according to BST rules (smaller to the left, larger to the right).
- Since the IDs were in ascending order, the BST became a right-skewed tree.
- Performed deletion operations for patient IDs 30, 70, and 50.
- Each deleted node had a single child, so the node was replaced directly by its child to maintain BST properties.
- Verified the final tree structure and calculated its height and lookup performance.
- Analyzed search complexity and concluded that while the current tree meets performance requirements, a self-balancing BST (AVL or Red-Black Tree) would provide better scalability.

Result : Screenshots/Output

```
BST Inorder after insertion:
```

```
20 30 35 40 45 50 60 65 70 75 80 85 90
```

```
BST Inorder after deleting 30, 70, and 50:
```

```
20 35 40 45 60 65 75 80 85 90
```

Time Complexity:

BST Construction: $O(n^2)$ (worst case due to sorted insertion)

- Search Operation: $O(n)$
- Delete Operation: $O(n)$
- The BST becomes right-skewed because patient IDs arrive in sorted order, reducing search efficiency.

GitHub Link :

Student Signature	Faculty Signature